Composition as Finite Tracking plus Gluing, and its Separation from Recursion in Abstract Indexed Programming Systems

Nova Spivack
https://www.novaspivack.com

March 2026

Preprint.

Archival version (Zenodo): https://doi.org/10.5281/zenodo.19429235
NEMS program hub (Zenodo): https://doi.org/10.5281/zenodo.19429271

Abstract

We study the relation between recursion and composition in abstract indexed programming systems. In standard computability-theoretic models these notions travel together, but at the level of abstract indexed partial semantics it is not a priori clear how much structure is really forced by the recursion theorem. We give two main results.
First, we prove an exact algebraic characterization of indexed representable composition:

$$\mathsf{I}^{\mathsf{idx}}_{\mathsf{comp}} \iff \mathsf{HasFiniteTracking} \wedge \mathsf{HasGluing}.$$

Thus composition splits cleanly into a *local* interpolation component (finite tracking) and a *global* amalgamation component (gluing). This sharpens the corrected exactness theorem

$$\mathsf{I}^{\mathsf{idx}}_{\mathsf{comp}} \iff \mathsf{SmnTrackingForRep}$$

by identifying the internal algebraic content of uniform tracking.
Second, we prove that indexed recursion does *not* imply indexed composition:

$$\exists \mathcal{A}\, \left(\mathsf{I}^{\mathsf{idx}}_{\mathsf{rec}}(\mathcal{A}) \wedge \neg \mathsf{I}^{\mathsf{idx}}_{\mathsf{comp}}(\mathcal{A})\right).$$

The countermodel sepAPS exhibits *section poverty under diagonal abundance*, as realized by the countermodel architecture: it has diagonal identity $\mathsf{smn}(x, x) = x$, fixed points for every indexed representable total unary map, and yet fails composition because its section family is too poor to realize a tracker for a distinguished nonconstant representable function g_{val}. The separation shows that pointwise fixed-point existence is strictly weaker than uniform parameterized tracking in abstract indexed systems.
These results resolve the recursion–composition frontier negatively in the abstract setting. The final separation theorem and its supporting realization are machine-checked in Lean 4 with zero additional axioms and zero `sorry`.
Trust boundary: Indexed separation and exactness claims are justified by the pinned Lean development; this PDF is exposition keyed to that artifact.
Archival version (Zenodo): https://doi.org/10.5281/zenodo.19429235

Keywords: acceptable programming systems, recursion theorem, s-m-n theorem, compositionality, uniform tracking, abstract computability, interface minimality, Lean 4 formalization

MSC 2020: 03D25, 03F40, 68Q05

Code/Data availability: Repository: https://github.com/novaspivack/aps-recursion-uniformization-lean.
Build: `lakebuild`.
Toolchain and module entry points are pinned in `lakefile.lean` and `MANIFEST.md` (repository root).

1 Premises, disagreement coordinates, and trust boundary

Where disagreement can (and cannot) live

Formal spine. The main separation and composition-anatomy theorems are justified by the Lean modules cited in this paper, not by informal roadmap claims alone.

Premise coordinates (for external readers).

- **Abstract indexed systems.** Countermodels live in the minimal `IndexedAPS` interface; standard computability models typically satisfy stronger hypotheses. *Logical effect:* Rejecting the abstract countermodel does not refute classical recursion theory on concrete models.
- **Living repository.** Declaration names and default targets evolve with repository tags. *Logical effect:* Use the artifact appendix and the pinned `lakefile.lean` for reproduction.

Contents

2 Introduction

Classical computability theory is usually developed inside concrete frameworks—Turing machines, partial recursive functions, acceptable numberings, or equivalent semantic codings [1, 2, 3, 4, 5]. In those settings, several structural phenomena appear tightly coupled: parameterization, composition, recursion/fixed-point theorems, Rice-style undecidability, and halting undecidability. Once one strips away the concrete implementation details and asks for the *minimal abstract structure* supporting each phenomenon, the picture becomes more delicate.

This paper concerns one specific frontier in that landscape: the relation between indexed recursion and indexed composition in abstract indexed programming systems. The central question is:

> Does the indexed recursion theorem force indexed representable composition?

For a substantial portion of the development, the evidence cuts in both directions. Standard models satisfy both properties. Several positive proof routes suggest that recursion should be rich enough to generate composition. At the same time, each such route encounters the same obstruction: the difference between pointwise self-reference and uniform parameterized tracking.

The final answer is negative. Indexed recursion does *not* imply indexed composition in abstract indexed programming systems. The paper proves this by combining an exact structural decomposition of composition with a concrete countermodel.

2.1 Main results

The paper establishes two main theorems. **Exact composition anatomy:** We identify the exact algebraic content of indexed composition:

$$\mathsf{I}^{\mathsf{idx}}_{\mathsf{comp}} \iff \mathsf{HasFiniteTracking} \wedge \mathsf{HasGluing}.$$

Here HasFiniteTracking says that every representable unary map can be tracked correctly on every finite set of parameters, while HasGluing says that such finite local trackers can be amalgamated into a single global tracker. **Strict separation from recursion:** We construct an indexed abstract programming system sepAPS such that

$$\mathsf{I}^{\mathsf{idx}}_{\mathsf{rec}}(\mathsf{sepAPS}) \quad \text{but} \quad \neg\mathsf{I}^{\mathsf{idx}}_{\mathsf{comp}}(\mathsf{sepAPS}).$$

Thus the recursion theorem is strictly weaker than representable composition in the abstract indexed setting. Together, these theorems resolve the recursion–composition frontier in the strongest possible way: composition is exactly analyzed, and recursion is shown not to force it.

Theorem 2.1 (Main separation theorem)**.** *There exists an indexed abstract programming system $\mathcal{A}$ such that*

$$\mathsf{I}^{\mathsf{idx}}_{\mathsf{rec}}(\mathcal{A}) \wedge \neg\mathsf{I}^{\mathsf{idx}}_{\mathsf{comp}}(\mathcal{A}).$$

Section 6 proves this theorem by constructing the countermodel sepAPS.

- **Exact composition theorem:** $\mathsf{I}^{\mathsf{idx}}_{\mathsf{comp}} \iff \mathsf{HasFiniteTracking} \wedge \mathsf{HasGluing}$.
- **Separation theorem:** $\exists\mathcal{A},\ (\mathsf{I}^{\mathsf{idx}}_{\mathsf{rec}}(\mathcal{A}) \wedge \neg\mathsf{I}^{\mathsf{idx}}_{\mathsf{comp}}(\mathcal{A}))$.
- **Interpretation:** Recursion is pointwise fixed-point existence; composition is uniform tracking. The classical coincidence of the two in standard models depends on additional structure not present in bare IndexedAPS.

Result	Meaning
$\mathsf{I}^{\mathsf{idx}}_{\mathsf{comp}} \iff \mathsf{SmnTrackingForRep}$	corrected exactness
$\mathsf{I}^{\mathsf{idx}}_{\mathsf{comp}} \iff \mathsf{HasFiniteTracking} \wedge \mathsf{HasGluing}$	algebraic decomposition
$\exists\mathcal{A},\ (\mathsf{I}^{\mathsf{idx}}_{\mathsf{rec}}(\mathcal{A}) \wedge \neg\mathsf{I}^{\mathsf{idx}}_{\mathsf{comp}}(\mathcal{A}))$	strict separation

The first two lines identify the exact internal anatomy of composition; the third shows that recursion remains strictly weaker even after that anatomy is exposed. This is the central structural lesson of the paper: exact decomposition on the composition side and strict non-derivability on the recursion side. The paper first identifies exactly what composition is, and then proves that recursion alone is not enough to force it.

2.2 Why the implication looked plausible

The negative resolution is not cheap or accidental. A substantial amount of structure supports the idea that recursion might imply composition.

(i) In standard computability models, recursion and composition coexist.

(ii) The sharp exactness theorem

$$\mathsf{I}^{\mathsf{idx}}_{\mathsf{comp}} \iff \mathsf{SmnTrackingForRep}$$

shows that composition is exactly uniform s-m-n-tracking for representable functions.

(iii) Several positive attack routes suggest that recursion might generate the missing uniformity: section surjectivity, finite tracking, Baire-category arguments on diagonal fibers, and failure-set uniformization.

What the final countermodel shows is that all of these positive routes were detecting a *real structural issue*, but not one forced by bare recursion itself. The key distinction is between:

- *diagonal abundance / fixed-point ease*, and
- *section richness / global tracking power*.

The countermodel has the former and lacks the latter.

2.3 Paper roadmap

The paper is organized as follows.

- Section 3 fixes the indexed setting and the main interface predicates.
- Section 4 recalls the corrected exactness theorem for composition and then proves the stronger algebraic decomposition

$$\mathsf{I}^{\mathsf{idx}}_{\mathsf{comp}} \iff \mathsf{HasFiniteTracking} \wedge \mathsf{HasGluing}.$$

- Section 5 explains why the implication looked plausible and summarizes the obstruction profile.
- Section 6 presents the countermodel sepAPS and proves

$$\mathsf{I}^{\mathsf{idx}}_{\mathsf{rec}}(\mathsf{sepAPS}) \wedge \neg \mathsf{I}^{\mathsf{idx}}_{\mathsf{comp}}(\mathsf{sepAPS}).$$

- Section 7 extracts the conceptual consequences: the recursion theorem is strictly weaker than composition, and the classical coincidence of the two in standard models depends on additional hidden structure.
- Section 8 records the machine-checked artifacts and repository layout.

3 Background: indexed abstract programming systems

We work throughout with indexed unary partial semantics. We retain the interface notation from the earlier formal development ($\mathsf{I}^{\mathsf{idx}}_{\mathsf{comp}}$, $\mathsf{I}^{\mathsf{idx}}_{\mathsf{rec}}$, $\mathsf{HasFiniteTracking}$, $\mathsf{HasGluing}$, etc.).

Definition 3.1 (Indexed abstract programming system)**.** An *indexed abstract programming system* is a structure

$$\mathcal{A} = (\varphi, \mathsf{smn})$$

where

$$\varphi : \mathbb{N} \to \mathbb{N} \rightharpoonup \mathbb{N}$$

assigns to each code e a partial unary function, and

$$\mathsf{smn} : \mathbb{N} \to \mathbb{N} \to \mathbb{N}$$

satisfies the s-m-n specification

$$\varphi_{\mathsf{smn}(e,x)}(n) = \varphi_e(\langle x, n \rangle) \qquad \text{for all } e, x, n.$$

Thus $\mathsf{smn}(e, x)$ is the x-section of the binary behavior coded extensionally by e.

Definition 3.2 (Indexed representable unary map)**.** A total unary map $h : \mathbb{N} \to \mathbb{N}$ is *indexed representable* in $\mathcal{A}$ if there exists e such that

$$\forall x,\ \varphi_e(x) = h(x).$$

We write $\mathsf{IndexedRepresentableUnary}(\mathcal{A}, h)$.

We next recall the main interface predicates.

Definition 3.3 (Indexed representable composition)**.** $\mathsf{I}^{\mathsf{idx}}_{\mathsf{comp}}(\mathcal{A})$ means: for every indexed representable $h : \mathbb{N} \to \mathbb{N}$, there exists k such that

$$\forall x\, n,\ \varphi_k(\langle x, n \rangle) = \varphi_{h(x)}(n).$$

Equivalently, a single code k uniformly tracks the family $x \mapsto h(x)$.

Definition 3.4 (Indexed recursion theorem)**.** $\mathsf{I}^{\mathsf{idx}}_{\mathsf{rec}}(\mathcal{A})$ means: for every $h : \mathbb{N} \to \mathbb{N}$, if $x \mapsto h(\mathsf{smn}(x, x))$ is indexed representable, then there exists e such that

$$\forall n,\ \varphi_e(n) = \varphi_{h(e)}(n).$$

Definition 3.5 (Indexed diagonal closure)**.** $\mathsf{I}^{\mathsf{idx}}_{\mathsf{diag}}(\mathcal{A})$ means: for every indexed representable $h : \mathbb{N} \to \mathbb{N}$, the diagonalized map

$$x \mapsto h(\mathsf{smn}(x, x))$$

is indexed representable.

The key conceptual point is immediate:

> $\mathsf{I}^{\mathsf{idx}}_{\mathsf{rec}}$ produces one fixed-point index e for each qualifying h. $\mathsf{I}^{\mathsf{idx}}_{\mathsf{comp}}$ produces one code k whose entire family of sections tracks $h(x)$ uniformly in x.

The question is whether the former already forces the latter.

3.1 Corrected exactness for composition

The corrected exactness theorem

$$\mathsf{I}^{\mathsf{idx}}_{\mathsf{comp}}(\mathcal{A}) \iff \mathsf{SmnTrackingForRep}(\mathcal{A}),$$

where

$$\mathsf{SmnTrackingForRep}(\mathcal{A})$$

means that every indexed representable unary map h admits a single k with

$$\forall x\, n,\ \varphi_{\mathsf{smn}(k,x)}(n) = \varphi_{h(x)}(n).$$

This gives the correct exact meaning of indexed composition: it is exactly uniform s-m-n-tracking for representable maps. The present paper strengthens this by decomposing that tracking property into local and global components.

4 Exact algebraic characterization of composition

We now isolate the two ingredients of uniform tracking.

4.1 Finite tracking

Definition 4.1 (Finite tracking)**.** $\mathsf{HasFiniteTracking}(\mathcal{A})$ means: for every indexed representable unary $h : \mathbb{N} \to \mathbb{N}$ and every finite set $F \subseteq \mathbb{N}$, there exists k such that

$$\forall x \in F\ \forall n,\ \varphi_{\mathsf{smn}(k,x)}(n) = \varphi_{h(x)}(n).$$

Thus finite tracking is local interpolation: every representable family can be tracked correctly on any prescribed finite parameter set.

4.2 Gluing

Definition 4.2 (Gluing)**.** $\mathsf{HasGluing}(\mathcal{A})$ means: whenever an indexed representable unary $h : \mathbb{N} \to \mathbb{N}$ has local trackers on every finite set, there exists a single k such that

$$\forall x\, n,\ \varphi_{\mathsf{smn}(k,x)}(n) = \varphi_{h(x)}(n).$$

In other words, gluing is the passage from finite local correctness to global correctness.

4.3 Main exactness theorem

The central positive theorem of Phase III is the following.

Theorem 4.3 (Composition as finite tracking plus gluing)**.** *For every indexed abstract programming system $\mathcal{A}$,*

$$\mathsf{I}^{\mathsf{idx}}_{\mathsf{comp}}(\mathcal{A}) \iff \mathsf{HasFiniteTracking}(\mathcal{A}) \ \wedge\ \mathsf{HasGluing}(\mathcal{A}).$$

(Lean: `comp_iff_finite_tracking_and_gluing`.)

Remark 4.4. The theorem separates the local content of composition (finite tracking) from the genuinely global content (gluing). In particular, any attempt to derive composition from recursion must explain where gluing comes from.

Proof idea. If $\mathsf{I}^{\mathsf{idx}}_{\mathsf{comp}}$ holds, then every representable unary map admits a global tracker k, which in particular works on every finite set; hence finite tracking holds. The same global tracker also witnesses gluing.

Conversely, assume finite tracking and gluing. Let h be indexed representable. Finite tracking gives a correct local tracker on each finite set $F \subseteq \mathbb{N}$. Applying gluing yields a single k correct on all of $\mathbb{N}$. This is exactly $\mathsf{I}^{\mathsf{idx}}_{\mathsf{comp}}$. □

Corollary 4.5 (Refined exactness)**.** *For every $\mathcal{A}$,*

$$\mathsf{I}^{\mathsf{idx}}_{\mathsf{comp}}(\mathcal{A}) \iff \mathsf{SmnTrackingForRep}(\mathcal{A}) \iff \mathsf{HasFiniteTracking}(\mathcal{A}) \ \wedge\ \mathsf{HasGluing}(\mathcal{A}).$$

This theorem is the conceptual heart of the positive side. It says composition is not primitive or opaque: it has an exact local–global anatomy. In particular, any implication from recursion to composition must explain not merely tracking, but why local tracking data can be globally amalgamated.

4.4 A surviving conditional positive theorem

Although recursion does not imply composition in general, one strong sufficient condition survives.

Theorem 4.6 (Conditional positive implication)**.** *For every $\mathcal{A}$,*

$$\mathsf{I}^{\mathsf{idx}}_{\mathsf{rec}}(\mathcal{A}) \ \wedge \ \mathsf{JointSectionSurjective}(\mathcal{A}) \ \wedge \ \mathsf{HasGluing}(\mathcal{A}) \ \Rightarrow \ \mathsf{I}^{\mathsf{idx}}_{\mathsf{comp}}(\mathcal{A}).$$

This theorem remains mathematically correct and useful. Its importance after the separation result is explanatory: it identifies exactly what extra structure (joint section surjectivity plus gluing) restores composition from recursion. Thus the separation theorem does not trivialize the positive side; it shows instead that joint section richness and gluing are genuine additional hypotheses, not consequences of bare recursion.

5 Why the false implication looked plausible

Before presenting the countermodel, it is worth explaining why the false implication looked plausible. The obstruction profile discovered by several positive routes all pointed in the same direction and all ran into the same structural barrier.

5.1 The pairing-shift obstruction

The most persistent obstacle is the gap between

$$\varphi_e(\langle x, n\rangle) \quad \text{and} \quad \varphi_e(n).$$

The *s-m-n* axiom gives access to sections of a code via paired input, but composition requires a code whose paired behavior realizes a target family $\varphi_{h(x)}(n)$. Repeatedly, positive proofs ran into the need to "undo" or transport across this pairing shift. In the abstract setting, that transport is not forced by recursion alone.

5.2 Section-surjectivity intuition

A natural idea is that recursion might force enough section richness to realize every extensional class as some section $k \mapsto \mathsf{smn}(k, x_0)$. If such section surjectivity held broadly enough, one could build finite trackers and then, with gluing, obtain composition.

This route never closed in the abstract setting. The later countermodel shows why: diagonal abundance need not produce section richness.

5.3 Baire-category and failure-set routes

Other attacks reformulated the problem topologically or combinatorially. In the Baire route, one studies fixed-point basins in diagonal fibers and asks whether topological largeness forces algebraic transport. In the failure-set route, one studies

$$F_k = \{x : [\mathsf{smn}(k, x)] \neq [h_0(x)]\},$$

hoping to prove enough intersection or regularity to force a common witness of failure or success.

These analyses were not wasted. The historical obstruction they discovered is no longer a missing proof obligation; it is part of the explanatory profile later realized by the countermodel. They correctly localized the obstruction profile:

- pairing shift,
- section poverty,
- local-vs-global uniformization,
- and the absence of forced gluing.

The countermodel shows that this obstruction profile is not merely a proof artifact. It is the actual separation mechanism.

6 Separation theorem

We now prove the main negative result. The proof has three steps: classify the indexed representable total unary maps in sepAPS (Theorem 6.5), verify fixed points for each of them (Theorem 6.6), and then show that g_{val} admits no global tracker (Theorem 6.7).

Theorem 6.1 (Recursion does not imply composition)**.** *There exists an indexed abstract programming system* sepAPS *such that*

$$\mathsf{I}^{\mathsf{idx}}_{\mathsf{rec}}(\mathsf{sepAPS}) \qquad \textit{and} \qquad \neg\mathsf{I}^{\mathsf{idx}}_{\mathsf{comp}}(\mathsf{sepAPS}).$$

(Lean: `I_rec_not_implies_I_comp`.)

Remark 6.2 (Contrast with standard computability models). In standard acceptable-numbering models, recursion and composition coexist because section richness, evaluation structure, and uniform tracking are built into the coding architecture; sepAPS shows that none of this is forced by the bare IndexedAPS axioms.

The rest of this section explains the construction and proof.

6.1 Architecture of the countermodel

The countermodel is infinite; earlier finite model searches did not detect the separation. The system sepAPS has three kinds of indices.

Index	Semantic behavior	Role
0	totally undefined	universal trivial fixed point
1	g_{val}	nonconstant self-sectioning function
$c+2$	constant-c function	all constant total functions

Proposition 6.3 (Diagonal identity in sepAPS)**.** *For all x,*

$$\mathsf{smn}(x,x) = x.$$

(Lean: `sep_smn_diag`.) This is the source of *diagonal abundance*: fixed points are easy to obtain.

Proposition 6.4 (Section poverty for g_{val})**.** *No index in* sepAPS *has paired behavior whose sections are undefined at $x = 0$ and defined at suitable nonzero parameters, and hence no index can track g_{val}.*

In particular, every section family in sepAPS is semantically uniform: either undefined everywhere or defined everywhere, whereas g_{val} demands undefined behavior at $x = 0$ and defined behavior at other parameters.

6.2 The function g_{val}

The nonconstant function $g_{\mathrm{val}} : \mathbb{N} \to \mathbb{N}$ (notation inherited from the formal development) is defined by recursion on the pair-tree structure of $\mathbb{N}$. Its crucial clauses are:

$$g_{\mathrm{val}}(0) = 0,$$

$$g_{\mathrm{val}}(\langle 0, n\rangle) = 0,$$
$$g_{\mathrm{val}}(\langle 1, n\rangle) = g_{\mathrm{val}}(n),$$
$$g_{\mathrm{val}}(\langle c+2, n\rangle) = c+2.$$

The decisive property is the self-sectioning law

$$g_{\mathrm{val}}(\langle 1, n\rangle) = g_{\mathrm{val}}(n).$$

This places index 1 on the diagonal in a way compatible with the *s-m-n* structure while keeping g_{val} nonconstant.

6.3 Representables in the model

Lemma 6.5 (Indexed representable total unary maps)**.** *A total unary map $h : \mathbb{N} \to \mathbb{N}$ is indexed representable in* sepAPS *(i.e.,* IndexedRepresentableUnary(sepAPS, h)*) iff either*

$$h = g_{\mathrm{val}} \qquad \text{or} \qquad h = \mathrm{const}_c$$

for some $c \in \mathbb{N}$.

(Lean: `sep_representable_classification`.)

Proof idea. Since φ_0 is nowhere defined, it does not represent any total unary map $h : \mathbb{N} \to \mathbb{N}$. Index 1 computes g_{val}. Each index $c+2$ computes the constant-c total map. No other total behaviors occur. □

6.4 Recursion theorem in the model

Lemma 6.6 (Every representable map has a fixed point)**.** $\mathsf{I}^{\mathsf{idx}}_{\mathsf{rec}}(\mathsf{sepAPS})$ *holds.*

Proof idea. By Theorem 6.5, every indexed representable total unary h is either g_{val} or a constant map.

Case $h = g_{\mathrm{val}}$: Take $e = 0$. Since $g_{\mathrm{val}}(0) = 0$, we have

$$\varphi_0 = \varphi_{g_{\mathrm{val}}(0)}.$$

Case $h = \mathrm{const}_c$: If $c = 0$, take $e = 0$. If $c = 1$, take $e = 1$. If $c \geq 2$, the constant map represented in sepAPS is coded by index c, whose semantic behavior is the constant-$(c-2)$ function. Choosing $e = c$ therefore gives $h(e) = e$, so $\varphi_e = \varphi_{h(e)}$. □

Thus recursion holds: every indexed representable total unary map has a fixed point.

6.5 Failure of composition

The negative side is witnessed by the nonconstant representable g_{val}.

Lemma 6.7 (No tracker exists for g_{val})**.** *There is no k such that*

$$\forall x\, n,\ \ \varphi_k(\langle x, n\rangle) = \varphi_{g_{\mathrm{val}}(x)}(n).$$

(Lean: `sep_no_I_comp`.)

Proof idea. Suppose such a k existed.

At $x = 0$, since $g_{\mathrm{val}}(0) = 0$ and φ_0 is nowhere defined, the tracker would have to satisfy

$$\varphi_k(\langle 0, 0\rangle) = \bot.$$

At $x = \langle 3, 0\rangle$, the value $g_{\mathrm{val}}(\langle 3, 0\rangle)$ lands in a defined constant class, so the same tracker would have to satisfy

$$\varphi_k(\langle\langle 3, 0\rangle, 0\rangle) = 1$$

(or the corresponding defined constant value in the formal indexing).

But every code in sepAPS has one of three globally uniform behaviors: nowhere defined, everywhere equal to g_{val}, or everywhere constant. No index mixes undefined behavior at one paired input with defined behavior at another. Contradiction. □

Corollary 6.8.

$$\neg \mathsf{I}^{\mathsf{idx}}_{\mathsf{comp}}(\mathsf{sepAPS}).$$

Combining Theorems 6.6 and 6.8 proves Theorem 6.1.

6.6 Interpretation of the separation mechanism

The countermodel exhibits a precise phenomenon:

section poverty under diagonal abundance.

Every index lies on the diagonal, so fixed points are easy. But the family of sections $\mathsf{smn}(k, x)$ is too poor to realize the mixed undefined/defined pattern required to track g_{val}. Thus recursion succeeds while composition fails.

This is exactly the structural mechanism that the failed positive routes were sensing in different languages.

Remark 6.9 (Intentional sparsity of sepAPS). The model is not an arbitrary pathological construction. It is engineered to maximize diagonal accessibility (every index on the diagonal) while starving the section family (no index mixes undefined and defined behavior). That design choice is precisely what realizes the separation. Its role is not to exhibit arbitrary pathology, but to isolate exactly the structural ingredients that recursion does *not* force. Its importance is therefore structural rather than anecdotal.

Remark 6.10 (Why finite countermodels did not separate). Phase II finite-model search (sizes $N = 2, \ldots, 5$) found no separation—the implication held in all finite APS models examined. The final countermodel is infinite. In finite models, the fixed-point reservoir is constrained; in sepAPS, the diagonal identity $\mathsf{smn}(x, x) = x$ and the infinite supply of constant indices provide enough structure for recursion without forcing section richness.

7 Consequences and interpretation

The separation theorem clarifies the abstract theory sharply.

7.1 Consequences for abstract indexed computability

The first consequence is immediate.

Corollary 7.1. *In abstract indexed programming systems,*

$$\mathsf{I}^{\mathsf{idx}}_{\mathsf{rec}} \not\Rightarrow \mathsf{I}^{\mathsf{idx}}_{\mathsf{comp}}.$$

Thus the recursion theorem and representable composition are genuinely distinct interface levels.

In standard computability models, section richness and evaluation structure are abundant. In sepAPS, diagonal abundance is maximized (every index lies on the diagonal), but section richness is minimized (no index mixes undefined and defined behavior). Therefore the standard coincidence of recursion and composition depends on additional structure not present in bare IndexedAPS.

7.2 Interpretation of the failed positive routes

The Baire route detected the temptation to infer section richness from basin largeness. The failure-set geometry detected the finite-intersection obstruction. The gluing route detected the local/global split inside composition. The countermodel validates these as real obstructions, not failed proof tricks. These attempted implication proofs were the correct structural diagnostics of a false implication:

- local and finite tracking are meaningful invariants;
- gluing is a genuine global principle;
- failure sets capture semantic mismatch at the level of extensional classes;
- the pairing shift is a real algebraic obstruction.

8 Machine-checked proof artifacts

The formal development is distributed across three repositories [8, 9, 10]. The final separation theorem and its supporting realization (`GValRealization.lean`, `Separation.lean`) are verified with zero additional axioms and zero `sorry`. Some historical exploratory modules (Baire-category attack, failure-set geometry) remain in the repository for explanatory and comparative value but are not load-bearing for the main results.

Code/Data availability: Lean 4 formalization: [8, 9, 10]. Verified by `lake update && lake exe cache get && lake build`.

9 Conclusion

The recursion–composition frontier in abstract indexed programming systems is now resolved.

Exact composition anatomy. Indexed composition is exactly finite tracking plus gluing:

$$\mathsf{I}^{\mathsf{idx}}_{\mathsf{comp}} \iff \mathsf{HasFiniteTracking} \wedge \mathsf{HasGluing}.$$

Strict separation from recursion. Indexed recursion does not imply indexed composition:

$$\exists \mathcal{A}\ (\mathsf{I}^{\mathsf{idx}}_{\mathsf{rec}}(\mathcal{A}) \wedge \neg \mathsf{I}^{\mathsf{idx}}_{\mathsf{comp}}(\mathcal{A})).$$

Conceptual lesson. The separation is driven by *section poverty under diagonal abundance*. The classical coincidence of recursion and composition in standard computability theory is not forced by the bare IndexedAPS axioms. What standard models add is precisely the section richness and global uniformization missing in the abstract countermodel.

Explanatory aftermath. The failed positive routes retain conceptual value. They identified, before the final countermodel was known, the exact obstruction profile that the separation realizes:

- pairing shift,
- section poverty,
- failure-set geometry,
- and gluing failure.

The final picture is therefore not merely negative but classification-level: the exact content of composition is known, and its non-derivability from recursion is witnessed concretely. Together, the exact decomposition of composition and the separation theorem show that the true missing structure between recursion and composition is not fixed-point existence itself, but the global uniformization encoded by gluing and section richness.

A Related Results from Earlier Phases

This appendix records several results established in earlier phases of the project that are not part of the main theorem spine of the present paper, but which provide useful structural context for the recursion–composition separation. They are omitted from the main body in order to keep the paper tightly focused on the exact algebraic decomposition of composition and the final separation theorem

$$\exists \mathcal{A}\ (\mathsf{I}^{\mathsf{idx}}_{\mathsf{rec}}(\mathcal{A}) \wedge \neg \mathsf{I}^{\mathsf{idx}}_{\mathsf{comp}}(\mathcal{A})).$$

Nevertheless, these earlier results remain mathematically significant and help situate the present paper inside the broader interface-minimality program.

A.1 Total-tier exactness for diagonal closure

Before the indexed partial setting, the project established an exact minimality theorem for diagonal closure in a total abstract programming system $\mathsf{APS} = (\mathsf{Program}, \mathsf{eval})$.

Definition A.1 (Total representable unary map)**.** A unary function $f : \mathbb{N} \to \mathbb{N}$ is representable in APS if there exists a program p such that

$$\forall x,\ \mathsf{eval}(p, x) = f(x).$$

Definition A.2 (Diagonal closure)**.** Fix $\mathsf{smn} : \mathbb{N} \to \mathbb{N} \to \mathbb{N}$. We say that APS has diagonal closure with respect to smn if

$$\forall h : \mathbb{N} \to \mathbb{N},\ \mathsf{RepresentableUnary}(\mathsf{APS}, h) \Rightarrow \mathsf{RepresentableUnary}(\mathsf{APS}, x \mapsto h(\mathsf{smn}(x, x))).$$

Theorem A.3 (Exact minimality for total diagonal closure)**.** *Assume* APS *has representable identity and representable composition. Then*

$$\mathsf{HasDiagonalClosureWrt}(\mathsf{APS}, \mathsf{smn}) \iff \mathsf{RepresentableUnary}(\mathsf{APS}, x \mapsto \mathsf{smn}(x, x)).$$

Thus, in the total tier, the diagonal map is not merely a convenient ingredient but the exact missing structure required for diagonal closure once composition and identity are present.

Remark A.4. This theorem belongs to a different layer from the indexed recursion–composition frontier studied in the main body. It is included here because it motivated the later indexed investigations: in the total tier, exactness is clean; in the indexed tier, the interaction among diagonalization, recursion, and uniform tracking turns out to be subtler.

A.2 Indexed separation lattice before the final separation theorem

Prior to the final countermodel sepAPS, the project established several strict separations in the indexed setting. These show that the interface landscape does not collapse even before the recursion–composition question is considered.

Theorem A.5 (Diagonal closure does not imply indexed composition)**.** *There exists an indexed abstract programming system $\mathcal{A}$ such that*

$$\mathsf{I}^{\mathsf{idx}}_{\mathsf{diag}}(\mathcal{A}) \qquad \textit{but} \qquad \neg\mathsf{I}^{\mathsf{idx}}_{\mathsf{comp}}(\mathcal{A}).$$

Theorem A.6 (Diagonal closure does not imply indexed recursion)**.** *There exists an indexed abstract programming system $\mathcal{A}$ such that*

$$\mathsf{I}^{\mathsf{idx}}_{\mathsf{diag}}(\mathcal{A}) \qquad \textit{but} \qquad \neg\mathsf{I}^{\mathsf{idx}}_{\mathsf{rec}}(\mathcal{A}).$$

Theorem A.7 (Weak recursion does not imply full recursion)**.** *There exists an indexed abstract programming system $\mathcal{A}$ such that*

$$\mathsf{WeakIndexedRecursion}(\mathcal{A}) \qquad \textit{but} \qquad \neg\mathsf{FullIndexedRecursion}(\mathcal{A}).$$

These results already showed that the abstract indexed interface hierarchy is genuinely stratified. The present paper strengthens that picture by proving that even full indexed recursion does not force indexed composition.

A.3 Corrected exactness and recursion taxonomy from Phase II

A central Phase II result was the corrected exactness theorem for indexed composition.

Theorem A.8 (Corrected exactness for indexed composition)**.** *For every indexed abstract programming system $\mathcal{A}$,*

$$\mathsf{I}^{\mathsf{idx}}_{\mathsf{comp}}(\mathcal{A}) \iff \mathsf{SmnTrackingForRep}(\mathcal{A}).$$

This theorem was the first sharp exactness result on the composition side. It shows that indexed composition is exactly uniform s-m-n-tracking for indexed representable unary maps.

Phase II also identified three distinct recursion notions:

- $\mathsf{WeakIndexedRecursion}(\mathcal{A})$: at least one qualifying map has a fixed point,
- $\mathsf{FullIndexedRecursion}(\mathcal{A})$: every qualifying map has a fixed point, i.e. $\mathsf{I}^{\mathsf{idx}}_{\mathsf{rec}}(\mathcal{A})$,

- $\mathsf{SmnTrackingRecursion}(\mathcal{A})$: every qualifying map admits a uniform s-m-n-tracker.

Theorem A.9 (Recursion taxonomy)**.** *Under indexed diagonal closure,*

$$\mathsf{SmnTrackingRecursion}(\mathcal{A}) \Rightarrow \mathsf{I}^{\mathsf{idx}}_{\mathsf{comp}}(\mathcal{A}) \Rightarrow \mathsf{I}^{\mathsf{idx}}_{\mathsf{rec}}(\mathcal{A}) \Rightarrow \mathsf{WeakIndexedRecursion}(\mathcal{A}),$$

and the final implication is strict.

The main paper resolves the remaining gap negatively by showing that $\mathsf{I}^{\mathsf{idx}}_{\mathsf{rec}} \not\Rightarrow \mathsf{I}^{\mathsf{idx}}_{\mathsf{comp}}$.

A.4 Rice bifurcation and mechanism bifurcation

Earlier phases also established a broader undecidability landscape, especially for Rice-style phenomena. This is not needed for the main recursion–composition theorem, but it is an important part of the surrounding interface program.

Theorem A.10 (Rice bifurcation)**.** *The abstract indexed Rice predicate splits into two mutually exclusive regimes:*

$$\mathsf{I}^{\mathsf{idx}}_{\mathsf{rice}} \Rightarrow \mathsf{VacuousRiceIndexed} \vee \mathsf{StrongRiceIndexed}.$$

Here:

- $\mathsf{VacuousRiceIndexed}$ means Rice truth holds because the representable Boolean fragment is too weak to realize nontrivial extensional deciders;
- $\mathsf{StrongRiceIndexed}$ means Rice truth holds in a genuinely structural sense, with nontrivial representable Boolean behavior present.

The structural regime itself was shown to bifurcate:

Theorem A.11 (Strong-Rice mechanism bifurcation)**.** *The structural Rice regime splits into at least two distinct inhabited mechanisms:*

(i) a fixed-point path, witnessed by standard computability models;

(ii) an alias path, witnessed by abstract systems with semantic aliasing but without full indexed composition.

Remark A.12. These bifurcation theorems are omitted from the main body because the present paper is about the recursion–composition frontier, not the full undecidability taxonomy. But they provide important evidence that abstract indexed systems admit a richer structural landscape than one might naively expect from standard computability theory alone.

A.5 Historical obstruction analyses and failed positive routes

Before the final countermodel was constructed, the project pursued several positive routes toward proving

$$\mathsf{I}^{\mathsf{idx}}_{\mathsf{rec}} \Rightarrow \mathsf{I}^{\mathsf{idx}}_{\mathsf{comp}}.$$

These routes are no longer active proof obligations, but they retain explanatory value because they correctly diagnosed the structural pressure points later realized by sepAPS.

Finite alias strategy. One idea was to produce a countermodel by adjoining finitely many aliases of a nonconstant extensional class. This fails because an adversarial h can map all finitely many aliases away from the target class simultaneously.

Infinite alias strategy. A countably infinite alias reservoir still does not automatically succeed: an adversarial h can move all aliases at once unless the model is engineered much more carefully.

Partial/divergence strategy. Another idea was to exploit divergence as a wildcard for fixed points. But the *s-m-n* specification tightly constrains divergence patterns: the section $\mathsf{smn}(e, x)$ is not an independent object, but is determined by the paired behavior of ϕ_e.

Restriction strategy. Restricting a standard system to a proper *s-m-n*-ideal also proved ineffective: once the ideal is rich enough to support the standard recursion argument uniformly, it tends to recover the same composition structure one was trying to destroy.

Baire-category route. A major historical route attempted to prove that nonmeager fixed-point basins in diagonal fibers would force section surjectivity and hence composition. The key intermediate statement $T6$ was eventually shown to be false. This was not wasted effort: it revealed that topological largeness does not by itself produce the algebraic transport needed to undo the pairing shift.

Failure-set route. Another route introduced the failure sets

$$F_k = \{x : [\mathsf{smn}(k, x)] \neq [h_0(x)]\}$$

and tried to prove enough intersection or regularity to force uniformization. The final outcome was that several local coherence properties do hold, but pairwise or finite intersection richness is not forced in bare IndexedAPS. This again turned out to be explanatory rather than merely frustrating: the countermodel sepAPS realizes exactly the kind of section poverty those analyses were detecting.

Remark A.13. The historical obstruction analyses belong to the explanatory aftermath of the final theorem. They show that the negative resolution is not ad hoc. The false implication looked plausible for good reasons, and the failed positive routes correctly located the real structural issue: recursion provides fixed-point abundance, but not the section richness and global amalgamation needed for composition.

A.6 How the present paper fits into the larger program

The larger project has three major layers.

(1) **Total-tier exactness:** exact minimality results for diagonal closure in total abstract programming systems;

(2) **Indexed interface landscape:** corrected exactness for composition, recursion taxonomy, separation lattice, and undecidability bifurcations;

(3) **Recursion–composition resolution:** exact algebraic decomposition of composition together with the final separation theorem

$$\exists \mathcal{A}\ (\mathsf{I}^{\mathsf{idx}}_{\mathsf{rec}}(\mathcal{A}) \wedge \neg \mathsf{I}^{\mathsf{idx}}_{\mathsf{comp}}(\mathcal{A})).$$

The present paper isolates the third layer and resolves it definitively. The related results recorded in this appendix show that the separation theorem does not stand in isolation: it sits inside a broader interface-minimality program in which exactness, bifurcation, and separation all play essential roles.

References

[1] S. C. Kleene. On notation for ordinal numbers. *Journal of Symbolic Logic*, 3(4):150–155, 1938. DOI: 10.2307/2267778.

[2] H. G. Rice. Classes of recursively enumerable sets and their decision problems. *Transactions of the American Mathematical Society*, 74(2):358–366, 1953. DOI: 10.2307/1990888.

[3] H. Rogers Jr. Gödel numberings of partial recursive functions. *Journal of Symbolic Logic*, 23(3):331–341, 1958. DOI: 10.2307/2964505.

[4] H. Rogers Jr. *Theory of Recursive Functions and Effective Computability*. MIT Press, Cambridge, MA, 1987.

[5] P. Odifreddi. *Classical Recursion Theory*. North-Holland, Amsterdam, 1989. Studies in Logic and the Foundations of Mathematics, vol. 125.

[6] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. In A. Platzer and G. Sutcliffe, editors, *Automated Deduction – CADE 28*, LNCS 12699, pages 625–635. Springer, 2021. DOI: 10.1007/978-3-030-79876-5_37.

[7] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, pages 367–381. ACM, 2020. DOI: 10.1145/3372885.3373824.

[8] N. Spivack. APS undecidability interfaces (Phase I). GitHub repository, 2024–2026. https://github.com/novaspivack/aps-undecidability-interfaces-lean. Accessed March 2026.

[9] N. Spivack. APS recursion composition uniformity (Phase II). GitHub repository, 2024–2026. https://github.com/novaspivack/aps-recursion-composition-uniformity-lean. Accessed March 2026.

[10] N. Spivack. APS recursion uniformization (Phase III). GitHub repository, 2024–2026. https://github.com/novaspivack/aps-recursion-uniformization-lean. Accessed March 2026.